# Seaglass Globe™

*The Agent's Presentation Surface — a Globe That Can Be Flown by Software*

A precision WGS-84 virtual globe with a native Model Context Protocol server: one `POST /mcp` endpoint on the running application, loopback-bound and bearer-protected in the suite install. Clients attach to a running globe rather than launching one — the user owns the window, and the agent joins it. The surface covers the camera, layers, projection, screenshots, scene entities, reports and raster overlays, military symbology and scenarios, and the full compute bridge: ForgeGIS™ operations, pipelines and batch runs; ForgeData™ catalog, extraction, elevation and publish. Results come back as things on the globe, not descriptions of them.

## /mcp

**one endpoint, one transport**

*MCP Streamable HTTP on the running app, loopback only; a stdio adapter for clients, ForgeMind™ included, that speak stdio*

## Scene

**sent with the question**

*since 1.2.5, the Assistant sends what is on the globe — entities with their symbol codes, draped overlays, the scenario clock — as structured context*

## Bounded

**every tool call**

*a wedged UI thread or an engine that never answers returns an error to the caller instead of hanging the agent — in 1.2.6 every engine call has a deadline; a panicking tool is caught, and the window stays up*

### Attach, don't spawn

The stdio adapter proxies to a running globe's `/mcp`; the desktop app serves that endpoint itself. An agent-launches-the-app model is wrong for an interactive viewer, so the design is the other way round: the port is bound before the window and the GPU come up, and readiness is observable — `tools/list`, then `list_ops` for ForgeGIS, then `list_datasets` for ForgeData. A tool called early answers `warming`, not an error.

### Results land on the globe

Every compute result is routed by shape. A summary goes to the report panel; a GeoJSON FeatureCollection becomes a scene entity; a raster handle is draped, reprojected by ForgeGIS, never guessed. A partial draw is reported as “drew N of M”, never as success — and a large result from a projected source is declined with a notice rather than drawn wrongly. `analyze_area` is the one-shot best-dataset → extract → compute → drape path.

### Answers that survive a restart

`publish_scene_entity` records an answer the operator can reference by name in later turns — site an HLZ as “Site 1”, then “route from Site 1”. When the ForgeData catalog is up it is written through, so the entity comes back after a restart with the same id. Since 1.2.5, `select_scene_entity` selects one for anchor editing, and `list_renderables` lists what the agent can toggle — feeds, agent geometry, heatmaps — with what each draws right now. `undo_last` undoes the last entity change from any source.

**Hardened in 1.2.6.** The server publishes an MCP contract — every listed tool with its description and input schema — answered by a hidden `contract_info` tool, even through the stdio adapter with the globe down. Each ForgeGIS and ForgeData child is checked against the contract the globe was built for when it spawns; a mismatch disables only the tools whose contract moved, each declined with both versions named. Every call to an engine has a deadline and is cancelled when it passes, while the bridge keeps serving, and a minimised globe's camera tools answer from its last visible size.


CONTINUED ON REVERSE →

WHO THIS IS FOR

## *Teams building on ForgeMind™*

The window you type into is the map that answers: ask in the Globe's Assistant and the answer is drawn on the globe; ask in ForgeGIS Studio™ and it lands on Studio's map. It is the same agent either way. The Assistant ships closed — View → Panels → Assistant docks it — and its bearer rides each request, never a client default.

## *MCP-native agent platforms and orchestrators*

Anything that speaks MCP over HTTP can drive the running globe — `initialize`, `ping`, `tools/list`, `tools/call`. Tool descriptions are hand-written agent-facing prose, because the description is what decides whether the tool gets picked. Every object-shaped result is also sent as `structuredContent`, so a client never has to pick a text block by position.

## *Builders of geospatial and exercise workflows*

Fly the camera, load a raster into an engine-side handle, run an operation or a saved pipeline against it, drape the result; browse the catalog, pull an extract, publish an answer back. Since 1.2.5, place MIL-STD-2525 symbols and tactical graphics, load and drive scenarios, and stream records into the scenario library over the same port. It is the same globe the operator sees, and a shape they draw by hand is the same kind of entity the agent places.

---

**Fire-and-queue, and now pollable.** Compute, catalog and image-overlay tools that queue work answer at once with `"status": "queued"`, `"done": false` and a `call_id`; in 1.2.6 `get_call_status` reads that id as queued, done or failed, with the messages the run reported and, if it failed, the error. `ok` still means accepted, not done, so an agent that reads it as success will look at an empty globe. Handles are opaque ids for rasters in the engine's memory, cleared when a child engine restarts.

---

**Read-back gaps, stated.** `list_renderables` now lists the toggleable renderables, but agents still can't read imagery or vector-file layer state back (`list_layers`), and `pick_at` and tour or bookmark tools are not on the surface yet. The loopback bind and the per-install bearer are the boundary, and in 1.2.6 every route also refuses a non-loopback `Host` and a cross-site browser request, so a DNS-rebinding page cannot reach it; a hand-launched globe with no token configured has no bearer, so never expose the port without setting one. Keep the window open and unminimised while an agent works: a minimised globe stops drawing — camera queries still answer from the last visible window — and drawing requests can time out.

## Build on the Globe

The Technical Brief covers the MCP surface, the compute and catalog bridges, the entity and scenario model, the digital-twin wire, and deployment; the generated tool reference ships in the suite documentation. Seaglass Globe is an opt-in component of the Forge suite, installed by the Seaglass Control Center™, and is not sold separately. The Forge suite is commercially licensed; evaluation builds are available to qualified teams.

**rich@seaglassfoundry.com · seaglassfoundry.com**

Seaglass Globe™, ForgeGIS™, ForgeMind™, ForgeData™, ForgeGIS Studio™, ForgeRelay™, Seaglass Control Center™, and Seaglass Foundry™ are trademarks of Seaglass Foundry LLC. Windows, DirectX, Vulkan, Rust, wgpu, and the Model Context Protocol are trademarks or marks of their respective owners; reference to them does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.